



WEB3

Cahier Des Charges : Web App TOUTATIX

2026

Auteur : Emilien MICARD

Table des matières

TABLE DES MATIERES	1
CONTEXTE	2
LA DEMANDE : API « ASTERIX VS OBELIX »	2
LIVRABLES	6
Maquettes	6
LES CONTRAINTES	6
Délais :	6
Contraintes techniques :	6
LES RESSOURCES	7
Effectifs :	7
Bibliothèques / Frameworks	7
IDE / Softs	7

Contexte

Dans le cadre de ce dernier module web, dans la continuité de tous les apprentissages acquis durant cette année et en anticipation du module de Machine Learning (BUT3), je vous propose la mise en œuvre d'un projet de groupe qui va faire écho aux APIs Web et au développement d'un client web lourd (ou riche).

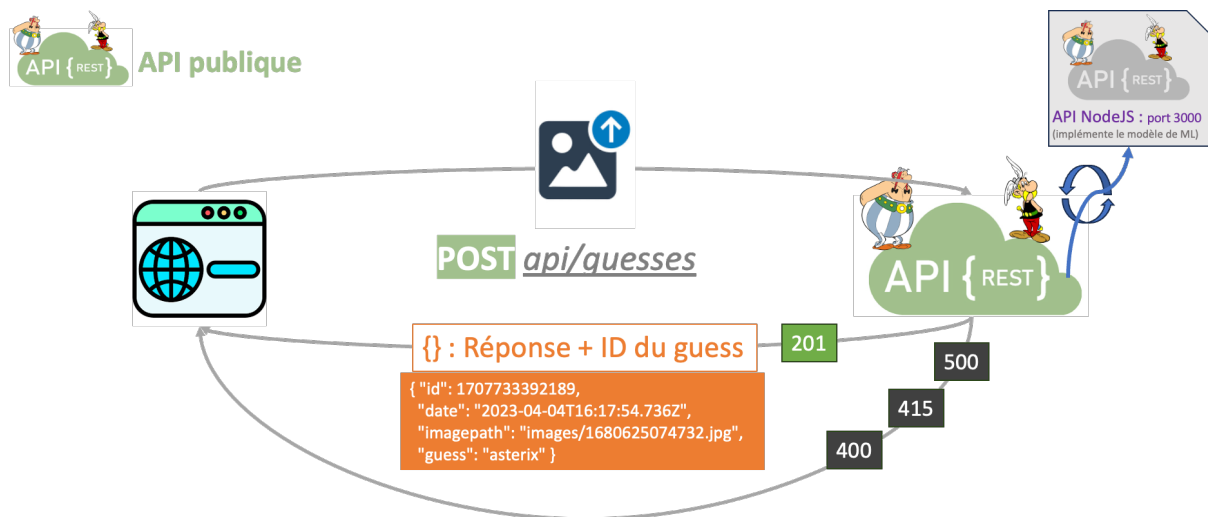


La demande : API « Astérix VS Obélix »

Pour ce projet, vous allez devoir réaliser une application web cliente de l'API « **Toutatix** » dont nous avons établis les spécifications plus tôt et que vous allez implémenter en parallèle dans notre module de Web2.

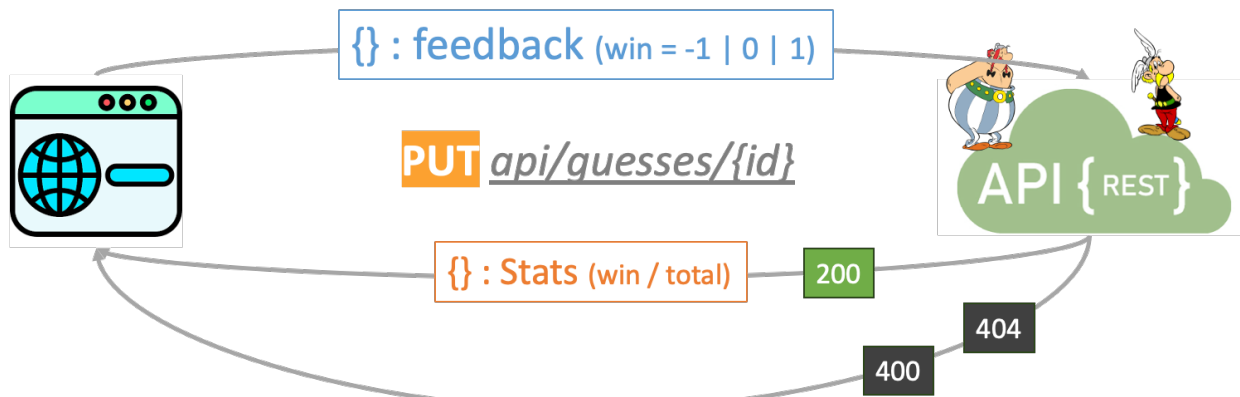
Pour rappel la facade « **Back-End** » de l'API (en **PHP**) utilise, un modèle de « **Machine-Learning** » développé sur **TensorFlow** ; Vous aurez tout loisir d'approfondir ces histoires de modèles en BUT3 mais pour le moment, vous vous contenterez d'utiliser ce modèle existant avec ses limites 😊

API publique : Use Case 1 : Faire une prédiction (guess) + Feedback



Un utilisateur public doit pouvoir **Poster** une **image** et, en retour, l'API, si tout se passe bien (*status 201*), retourne en réponse **la prédiction (Asterix / Obélix)** ainsi qu'un **ID** pour cette prédiction (*utile pour faire le feedback – cf. ci-après*)

Pour se faire, vous disposerez d'une **API tierce** à inclure dans votre « stack docker » et qu'il vous faudra « **appeler** » pour réaliser la prédiction. *Bien sûr, ce détail d'implémentation ne devra pas apparaître dans votre spécification Swagger* 😊



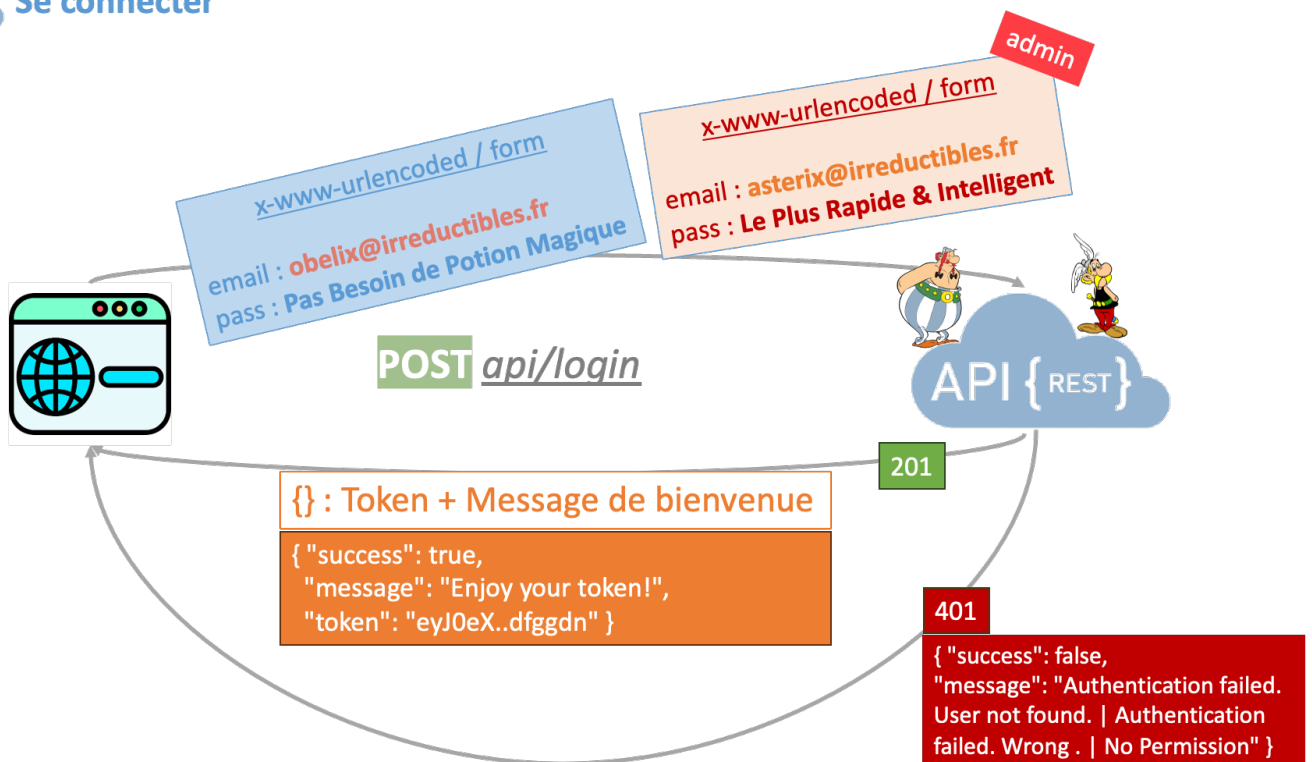
Afin d'améliorer le modèle par la suite, l'API doit permettre de donner un **feedback** sur la prédiction faite. Aussi l'utilisateur ayant demandé la prédiction doit réaliser un **PUT** sur la précédente prédiction (**ID du guess**) afin d'indiquer si elle était **bonne (win = 1)**, **mauvaise (win = -1)** ou s'il ne s'agissait en fait **ni d'Asterix, ni d'Obélix (win = 0)**.

En retour l'API, si tout est **OK (status 200)**, retourne les stats de l'IA, cad. le nombre de bonne prédictions sur le total.

API protégée : Use Case 2: Connexion



Se connecter

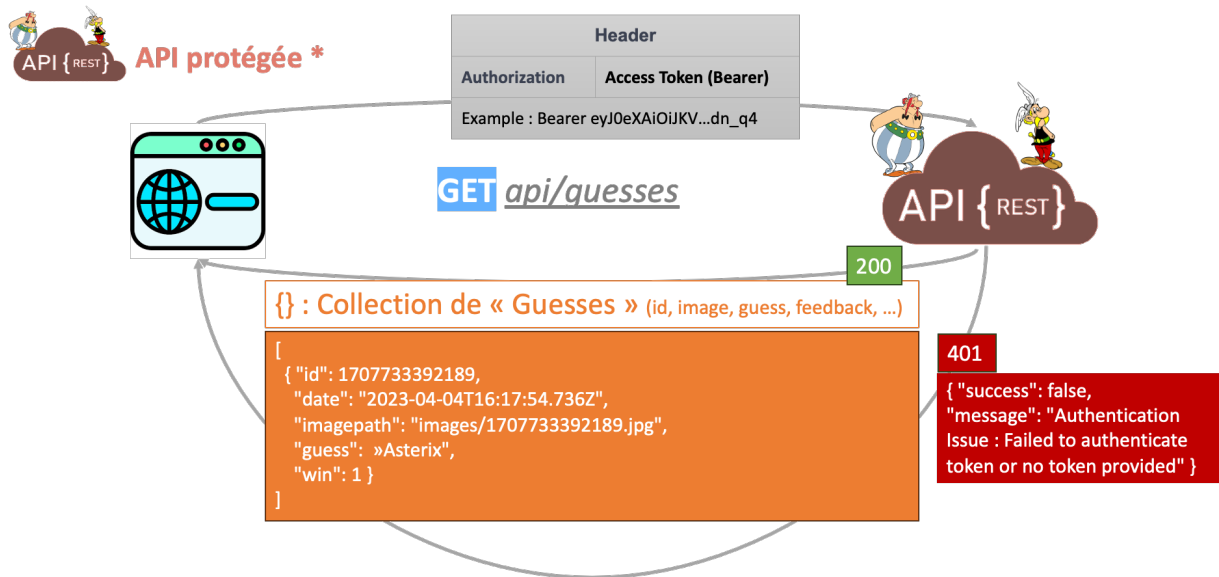


Pour pouvoir accéder aux méthodes protégées, il va falloir se **connecter** et **récupérer un token**. C'est bien à cela que va servir la méthode de **login**.

Pour se faire, il faut **Poster les identifiants** (sous forme de champs de formulaire : `x-www-urlencoded`) et si ceux-ci sont reconnus, **l'API retourne le token**.

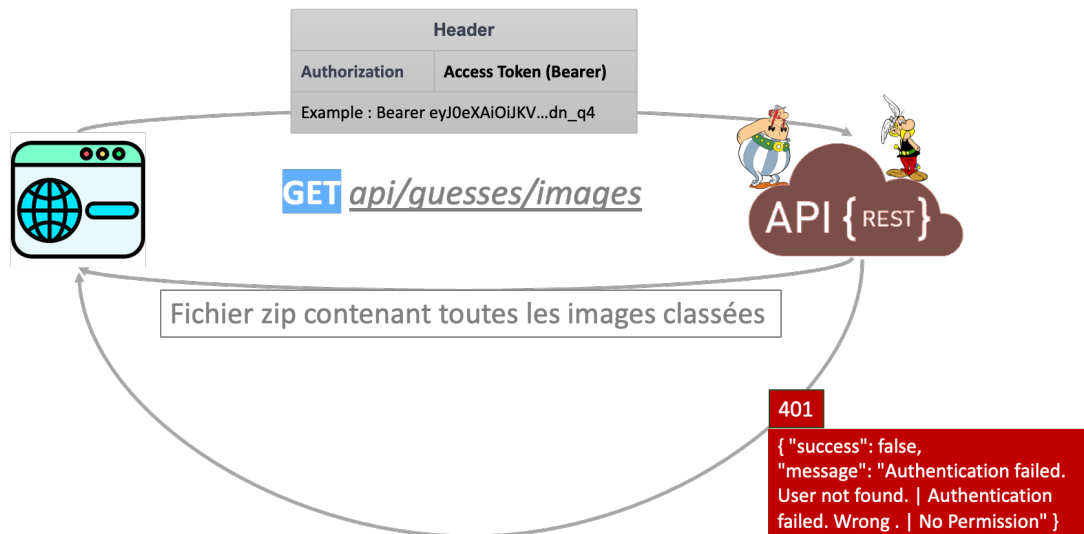
Sinon, l'API répond avec le **status 401** et un message json explicatif.

API protégée : Use Case 2 : Historique & Extraction



Cette méthode **protégée** (donc nécessitant le passage du token dans le header) permet de récupérer **l'historique de toutes les prédictions** (guesses) avec les **feedbacks** si ceux-ci ont été renseignés.

API protégée *

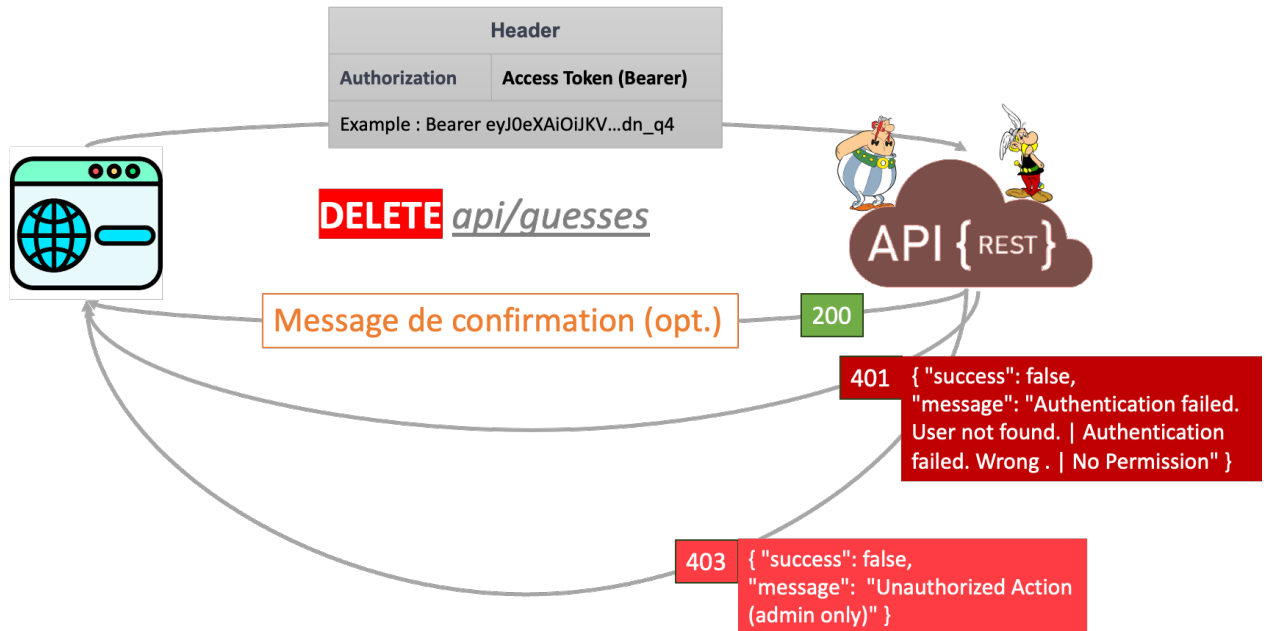


Cette autre méthode protégée permet de **télécharger dans un ZIP toutes les images des prédictions classées par sujet (Asterix / Obelix)** et par **bonne/mauvaise prédiction (win = 1 / win = -1)**, les images sans feedback ne sont pas téléchargées. Ceci permettant de pouvoir améliorer le modèle par la suite.

API protégée : Use Case 3 : Réinitialisation



API protégée *



Cette dernière route, **uniquement accessible à l'utilisateur « admin »** (sinon : *status 403*), permet de **supprimer tout l'historique et toutes les images postées**.

Livrables

Pour ce mini projet, il vous sera demandé **2 livrables** :

- Des maquettes d'écran (Mockup) ⇔ figma ? 
- Le client web déployé (pages HTML/CSS/JS) 

Maquettes

Votre appli web finale devra contenir 3 pages (3 onglets) :

Défiiez moi !	Historique	Stats
Page 1 : Accès public : Défiiez moi ! Cette page permettra de poster une photo à l'API pour obtenir la réponse de l'IA. Et de donner le feedback / la validité de la réponse. 	Page 2 : Accès protégé : Historique Cette page, à usage uniquement des utilisateurs connectés, permettra de visualiser la liste de tous les « guesses » (cf API json objet guess) + possibilité de télécharger toute les images 	Page 3 : Stats Affichez un ou plusieurs graphes des stats de l'IA. Par exemple un "stack chart" avec le nombre de prédictions totales, réussies, ratées, invalides. 

Les contraintes

Délais :

- J'attends vos maquettes finalisées (avec Audrey B. normalement) pour **le 26 mai**
- Vous ferez une démonstration de votre projet **le 9 juin**.

Contraintes techniques :

Votre appli web devra :

- Être compatible multi-support (Mobile, Desktop, Tablette) → **Responsive**
- Fonctionner de manière **Asynchrone** (pas de blocage pendant les appels API)
- Tourner sur navigateur web (Desktop et Mobile)** → **Techno HTML/CSS/JS**
- Être en mesure d'utiliser la **Caméra de votre périphérique mobile**.

Les ressources

Effectifs :

Vous réaliserez ce projet par **groupe de 4 personnes**. Essayez de vous répartir les tâches intelligemment en fonction de vos compétences et de vos groupes TP:

- Design
- Code d'appel à l'API

Et des fonctionnalités :

- Page « Défier l'IA »
- Pages d'historique et de stats

Bibliothèques / Frameworks

En plus des technos de bases imposées par les contraintes techniques (HTML/CSS/JS), vous aurez le loisir d'utiliser les **Frameworks** et **Bibliothèques** que vous voulez.

Vous pouvez bien évidemment utiliser **Bootstrap** comme dans nos précédents TPs si vous le souhaitez.

En outre, je peux vous conseiller d'utiliser **getUserMedia()** pour la capture photo depuis la caméra de votre mobile et **Chart.JS** pour le(s) graphe(s).

IDE / Softs

En termes d'IDE, là encore je ne vous impose rien (*même avec VI pour les plus radicaux 😊*) mais vu que vous avez l'habitude de bosser avec **VSCode**...

Enfin, je vous invite fortement à utiliser **Postman** ou similaire afin de tester les requêtes sur l'API.